

Herald



Herald — qaherald Flavor Guide (Operator)

Field	Value
Revision	1
Created	2026-05-30
Last modified	2026-05-30
Status	active
Status summary	Nano-detail operator reference for qaherald (QA Herald) — a CLI-only flavor (DefaultPort=0) that is Herald's autonomous QA bot. Documents <code>run</code> (scenario harness → docs/qa evidence), <code>mtproto</code> (the MTProto user-client session lifecycle: <code>login/whoami/logout</code>) and WHY MTProto is required (Telegram's bot-privacy boundary), <code>lifecycle</code> (the 15-scenario lifecycle driver — currently a SKELETON), and <code>version</code> . ANTI-BLUFF: derived from the built qaherald binary (<code>qaherald --help</code> , per-subcommand <code>--help</code> , <code>version --json</code>) + <code>commons/branding.go</code> ; the lifecycle SKELETON status is quoted verbatim from the binary's own help.
Issues	(none specific to this guide)
Continuation	Bump when <code>lifecycle T2</code> (messenger) + <code>T3</code> (scenarios) wire the real driver and when live MTProto evidence lands under <code>docs/qa/</code> .

Table of contents

- [§1. What qaherald is](#)
 - [§2. The subcommand surface](#)
 - [§3. version](#)
 - [§4. Why MTProto \(and not the Bot API\)](#)
 - [§5. mtproto — the user-client session lifecycle](#)
 - [§6. run — the scenario harness](#)
 - [§7. lifecycle — the 15-scenario driver \(skeleton\)](#)
 - [§8. References](#)
-

§1. What qaherald is

qaherald is **QA Herald** — flavor key **qa**, prefix **QHR**, **CLI-only** (DefaultPort=0 — it drives external services, it does not serve HTTP itself). Per `commons/branding.go` its mission is "QA bot — pherald [?] Telegram round-trip automation + docs/qa/ evidence". It is the binary that satisfies Herald's §107.x evidence mandate: it drives pherald [?] Telegram round-trips end-to-end, records bidirectional transcripts + content-addressed attachments under `docs/qa/<run-id>/`, and emits a Markdown report. Per HelixConstitution §11.4.98 (Full-Automation Anti-Bluff Mandate), it must do this **fully autonomously** — which is exactly why it speaks MTPROTO.

Build it:

```
go build -o /tmp/qaherald ./qaherald/cmd/qaherald
```

§2. The subcommand surface

Verbatim from `qaherald --help`:

Subcommand	What it does	State
<code>run</code>	Run qaherald scenarios against the configured pherald + Telegram bot	live
<code>mtproto</code>	Manage the MTPROTO user-client session used by qaherald (<code>login/whoami/logout</code>)	<code>login</code> interactive; <code>whoami/logout</code> autonomous
<code>lifecycle</code>	Run the 15-scenario lifecycle test against pherald <code>listen</code> via a 2nd Telegram bot	SKELETON — driver not wired (T2/T3)
<code>version</code>	Print QA Herald version + build info	live
<code>completion</code>	Generate shell autocompletion (Cobra built-in)	live

§3. version

```
$ qaherald version --json
{"arch":"arm64","binary":"qaherald","build_time":"unknown","commit":"unknown-dev"}
```

§4. Why MTPROTO (and not the Bot API)

Quoted from `qaherald mtproto --help`: Telegram's bot privacy boundary blocks a bot from observing another bot's messages in non-DM contexts. The QA flavor **MUST** drive the **system** bot (the one pherald `listen` runs) end-to-end; the only autonomous way to do that is via a **real user account** speaking MTPROTO. A second bot cannot see the first bot's group messages, so a bot-to-bot QA loop is a dead end — hence qaherald carries an MTPROTO user-client.

This is consistent with §11.4.98: a test that needs a human to type a Telegram message is a **PASS-bluff** at the automation layer. The MTPROTO user-account is the autonomous driver that removes the human from the loop.

§5. mtpROTO — the user-client session lifecycle

`qaherald mtpROTO` manages the persisted MTPROTO user-client session. Lifecycle:

1. **One-time bootstrap (interactive):** `qaherald mtpROTO login` — the operator enters Telegram's SMS / app-push code. This is the single permitted manual step (credential bootstrap, not test-driving — §11.4.98's allowed exception).
2. **Health check (autonomous):** `qaherald mtpROTO whoami` — prints the connected user identity.
3. **Teardown / rotate (autonomous):** `qaherald mtpROTO logout` — server-side LogOut + removes the local session file.

Required environment (all four):

Variable	Source / meaning
<code>HERALD_MTPROTO_APP_ID</code>	From https://my.telegram.org/apps (integer)
<code>HERALD_MTPROTO_APP_HASH</code>	From https://my.telegram.org/apps (32-char hex)
<code>HERALD_MTPROTO_PHONE</code>	E.164 phone of the QA user account
<code>HERALD_MTPROTO_PASSWORD</code>	Cloud 2FA password (empty if 2FA off)

The session file lives at `~/ .config/herald/mtpROTO.session` (mode 0600). It is never committed, never echoed, and protected at rest only by filesystem ACL.

```
export HERALD_MTPROTO_APP_ID=...
export HERALD_MTPROTO_APP_HASH=...
export HERALD_MTPROTO_PHONE=+15551234567
qaherald mtpROTO login      # interactive — enter the Telegram
                             code once
qaherald mtpROTO whoami     # autonomous health check
```

SAFETY: the MTPROTO path uses a real Telegram account. Follow the credential-setup guidance in `docs/guides/OPERATOR_CREDENTIALS.md` (and the Telegram-official cross-references there) — using a VoIP number or skipping the recovery-email setup risks an account ban.

§6. run — the scenario harness

`qaherald run` drives the configured set of scenarios end-to-end: constructs CloudEvents, POSTs them via HTTPS+JWT (with TOON content negotiation per Wave 4b), observes Telegram-side delivery, records a bidirectional transcript + content-addressed attachments under `docs/qa/<run-id>/`, and emits a Markdown report. Per §107.x it is the canonical producer of the `docs/qa/<run-id>/` evidence artefact.

Exit code is the contract: 0 when every scenario PASSes, non-zero on any FAIL — a silent FAIL would be a §11.4 PASS-bluff at the harness layer.

Flag	Env	Meaning
<code>--herald-url</code> string	<code>HERALD_BASE_URL</code>	pherald base URL (default <code>https://localhost:7443</code>)
<code>--herald-token</code> string	<code>HERALD_JWT_SECRET</code>	JWT HMAC secret for pherald — REQUIRED

Flag	Env	Meaning
<code>--tg-token</code> string	HERALD_TGRAM_BOT_TOKEN	Telegram bot token — REQUIRED
<code>--tg-chat</code> int	HERALD_TGRAM_CHAT_ID	Telegram chat ID — REQUIRED
<code>--scenario</code> string	—	Scenario name (e.g. happy-path-single-channel) or all (default all)
<code>--scenario-timeout</code> duration	—	Per-scenario timeout (default 1m)
<code>--out-dir</code> string	—	Parent directory for the <run-id>/subdirectory (default docs/qa)

```
qaherald run --scenario all \
  --herald-url https://localhost:7443 \
  --herald-token "$HERALD_JWT_SECRET" \
  --tg-token "$HERALD_TGRAM_BOT_TOKEN" --tg-chat 987654321
```

§7. lifecycle — the 15-scenario driver (skeleton)

`qaherald lifecycle` automates the S01..S15 scenarios from `tests/test_wave6.5_lifecycle.sh` by posting each input via a 2nd Telegram bot (HERALD_QA_BOT_TOKEN) and asserting pherald's reply + filesystem mutation. Outputs land under `docs/qa/HRD-101-lifecycle-<run-id>/` (`transcript.jsonl`, `report.md`, content-addressed attachments/).

State: SKELETON only. Quoted verbatim from the binary's help: "T1 status: SKELETON only — T2 (messenger) and T3 (scenarios) wire the actual driver. Running this command today resolves env fallbacks, validates required flags, creates the OutDir, and exits with a 'not-wired-yet' message." Do not rely on `lifecycle` for real QA evidence yet — use `run` (§6).

Pre-reqs documented by the help (for when T2/T3 land): `pherald listen` running with `--qa-out-dir` set; the qa-bot in the same group as the pherald-bot; the qa-bot's Privacy Mode DISABLED (via `@BotFather` → `/setprivacy` → `Disable`); `HERALD_OPERATOR_IDS` containing the qa-bot's user-id. Notable flags include `--qa-bot-token` (env `HERALD_QA_BOT_TOKEN`), `--chat-id`, `--scenarios` (subset of S01..S15), `--run-id`, and `--skip-preflight` (DEV ONLY — forbidden in prod). Run `qaherald lifecycle --help` for the full set.

§8. References

- Source: `qaherald/cmd/qaherald/main.go`, `mtproto_cmd.go`, `run + lifecycle` command files.
- Branding: `commons/branding.go` (flavor `qa`, `DefaultPort=0`).
- Evidence mandate: Herald §107.x (docs/qa evidence) + HelixConstitution §11.4.98 (full-automation anti-bluff).
- MTPROTO credentials: `docs/guides/OPERATOR_CREDENTIALS.md` (MTPROTO bootstrap + Telegram-official safety cross-references); <https://my.telegram.org/apps> (app id/hash source).
- Companion flavor guides: `docs/guides/{PHERALD, SHERALD, CHERALD, BHERALD, RHERALD, IHERALD, SCHERALD}.md`.

Sources verified

Verified 2026-05-30: Telegram developer portal — <https://my.telegram.org/apps> (the source of HERALD_MTPROTO_APP_ID / HERALD_MTPROTO_APP_HASH, cross-referenced because qaherald is a risk-classified Telegram-touching tool). All subcommand, flag, default, env-var, and state claims (lifecycle SKELETON, mtpROTO login interactive vs whoami/logout autonomous, the MTPROTO-vs-Bot-API rationale) were derived by running the built qaherald binary (qaherald --help; qaherald run|mtpROTO|mtpROTO login|lifecycle --help; qaherald version --json) and reading commons/branding.go on 2026-05-30 — the SKELETON status is quoted verbatim from the binary's own help. No flags were invented. **Re-verification cadence (per §11.4.99 (C)):** Telegram is a risk-classified service → **90-day staleness** (next due **2026-08-28**) for the my.telegram.org app-id/hash + account-safety guidance; the rest tracks the qaherald Cobra surface — re-verify on any subcommand change (notably when lifecycle T2/T3 land).